

How University Students Read Non-English Code

Rachel Wilson

Carnegie Mellon University

Motivation

Computer programming is English-centric, meaning non-native speakers face barriers when learning to program [2].

Compare for yourself the following code.

```
18 定义 转换输出(数目):      def transformOutput(num):
19     结果 = 1                result = 1
20     循环 (数目 // 100) 里的:  for i in range(num//5):
21         结果 = 结果 * 3      |     result = result * 2
22     返回 结果              return result
23
```

To address this issue, one might ask:

Should programming languages be translated or should programmers learn English?

Research Focus

In this case study, I research how English-speaking university students leverage their diverse linguistic and programming backgrounds to read non-English code.

Goals

1. Identify what skills students use to read non-English code
2. Formulate further research questions
3. Reflect on localized vs. English programming languages

Approach

First, understand linguistic and programming backgrounds through a **survey**.
Next, observe how students read non-English code in **interviews with a coding activity**.

Delay with One Sender and Multiple Hops

Setup

Links have capacities, c_i , that determine its sending speed.

How long does it take p packets of b bits each to traverse the network?
It's not obvious how long packets spend on queues.

Result for One Sender

Suprisingly, the order of links does not matter. **End-to-end latency only depends on the slowest link with capacity $c_{\min}$.**

$$\text{Total time} = \frac{(p-1)b}{c_{\min}} + \sum_i \frac{b}{c_i}$$

Proof Sketch

Between two links, who finishes first? Does time spent per packet change?

Case 1: $w_{i-1} + \frac{b}{c_{i-1}} \geq \frac{b}{c_i} \rightarrow$ No change in time between packets

Case 2: $w_{i-1} + \frac{b}{c_{i-1}} < \frac{b}{c_i} \rightarrow$ Time increases to the rate of the slower link

Across all links, the time per packet is stretched to the slowest rate.

Delay with Multiple Senders and Multiple Hops

Challenge

When multiple senders share bandwidth, it's hard to reason deterministically about the interleaving of their packets. How do we solve this?

Poisson Processes

Poisson processes model arrival rates λ and link capacities c . If stable ($\lambda < c$), they have nice properties [1]:

- If the input is Poisson(λ), so is the output.
- Poisson(λ_1) + Poisson(λ_2) = Poisson($\lambda_1 + \lambda_2$)

Result for Multiple Senders

When stable, $\mathbb{E}[T_{\lambda_i}]$ is the average time a packet from λ_i spends in the network [1].

$$\mathbb{E}[T_{\lambda_1}] = \mathbb{E}[T_{\lambda_2}] = \frac{1}{c_1 - (\lambda_1 + \lambda_2)} + \frac{1}{c_2 - (\lambda_1 + \lambda_2 + \lambda_3)}$$
$$\mathbb{E}[T_{\lambda_3}] = \frac{1}{c_2 - (\lambda_1 + \lambda_2 + \lambda_3)}$$

Conclusion and Next Steps

We can now easily model delay for multiple senders over multiple hops. It remains difficult to integrate this into a game theoretic framework.

Towards integration with game theory, we must assess the stability assumption and bound the delay so it does not depend on subsets of senders' rates. This will ultimately provide minimum throughput guarantees for senders.

References

- [1] Mor Harchol-Balter. *Performance modeling and design of computer systems: queueing theory in action*. Cambridge University Press, 2013.
- [2] Pratiksha Thaker, Matei Zaharia, and Tatsunori Hashimoto. Don't hate the player, hate the game: Safety and utility in multi-agent congestion control. In *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks*, pages 140–146, 2021.